

Informe de Reflexión Técnica y Métricas DORA

Proyecto: inventario-app | Entorno: CI/CD GitHub Actions & Kubernetes (Minikube)

1. Justificación Técnica de la Estrategia de Despliegue (Canary)

Para la aplicación **inventario-app** se seleccionó e implementó la estrategia **Canary Deployment** (configurada en `k8s/canary/` con 4 réplicas para `v1` [80%] y 1 réplica para `v2` [20%]). Esta elección se justifica técnicamente por las siguientes razones para esta arquitectura REST de catálogo:

- **Eficiencia de Recursos:** A diferencia de *Blue-Green*, que exige duplicar al 100% la infraestructura de cómputo en el clúster (200% de capacidad requerida durante el despliegue), Canary permite introducir la nueva versión asignando únicamente un 20% de capacidad adicional o redistribuyendo réplicas existentes.
- **Mitigación Gradual del Riego de Errores:** En una API REST de inventario, si una nueva versión introduce una regresión o fuga de memoria, en Blue-Green el 100% de los usuarios experimentarían el fallo al conmutar. En Canary, únicamente el 20% del tráfico se ve expuesto inicialmente, permitiendo detectar anomalías en métricas antes de continuar la promoción.
- **Simplicidad de Rollback:** Si la réplica Canary responde errores o eleva su latencia, el rollback se efectúa eliminando simplemente el Deployment `v2` en segundos, sin requerir reconfiguraciones complejas en el Service.

2. Observación de Persistencia de Datos al Recrear Pods (Paso 5)

Durante las pruebas prácticas sobre Minikube, se creó un producto desde la interfaz web (almacenado en `data/products.json`) y posteriormente se forzó la eliminación del Pod ejecutor mediante `kubectl delete pod`. Al recrearse el Pod, **los datos creados desaparecieron**. El diagnóstico técnico de este comportamiento es:

- **Almacenamiento Efímero:** La base de datos `db.js` guarda la información directamente en el sistema de archivos del contenedor local. Los contenedores en Kubernetes son inmutables y efímeros; al destruirse el contenedor, su capa de escritura se elimina por completo.
- **Solución de Producción:** Para resolver este comportamiento en entornos reales, la aplicación requeriría un volumen persistente (`PersistentVolumeClaim` de Kubernetes montado en `/app/data`) o preferiblemente migrar hacia una base de datos externa desacoplada (PostgreSQL, MongoDB, Redis).

3. Métricas DORA Propias y Nivel de Desempeño

A partir de los registros de commits de Git y las ejecuciones en GitHub Actions durante el desarrollo del laboratorio, se calcularon las tres métricas DORA solicitadas:

Métrica DORA	Valor Calculado	Clasificación DORA	Detalle y Evidencia
Lead Time for Changes	12.5 minutos	High Performance	Tiempo medio entre el commit local y la confirmación de corrida exitosa en Minikube (Commit A: 14m, Commit B: 11m).
Deployment Frequency	4 despliegues / día	High Performance	8 despliegues promovidos al clúster en un período de 2 días de laboratorio.

Change Failure Rate	12.5%	High Performance	1 fallo de despliegue (probe timeout por arranque lento) de 8 despliegues totales requeridos.
----------------------------	-------	-------------------------	---

4. Bitácora de Problemas Reales Encontrados y Solución

- **Problema 1 — Sensibilidad a Mayúsculas en Dockerfile (Linux CI Runner):**

Síntoma: En Windows el archivo ``dockerfile`` (minúscula) funcionaba en local, pero el runner de GitHub Actions (Ubuntu Linux) fallaba buscando ``./Dockerfile``.

Solución: Se ejecutó `git mv dockerfile Dockerfile` para registrar la mayúscula de forma consistente en el repositorio remoto.

- **Problema 2 — Pod Killeado por Readiness Probe en Arranque Lento:**

Síntoma: Al simular `STARTUP_DELAY_SECONDS=10`, Kubernetes marcaba el Pod como ``Unhealthy`` y procedía a reiniciarlo en bucle infinito (CrashLoopBackOff).

Solución: Se reconfiguró el readinessProbe en `k8s/deployment.yaml` extendiendo `initialDelaySeconds: 5`, `periodSeconds: 3` y `failureThreshold: 5`, otorgando un margen de 17 segundos antes de fallar.

- **Problema 3 — Fallo de Autenticación en Publicación a GHCR:**

Síntoma: El job ``build-push`` devolvía ``denied: permission_denied`` al intentar publicar la imagen Docker.

Solución: Se configuró la sección `permissions: packages: write` en `.github/workflows/ci-cd.yml` autorizando al `GITHUB_TOKEN` nativo de la acción.